

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CSA0504 — Database Management Systems

Assignment

Assignment Title:

Smart National Exam Slot Booking and Result Management System

Course Outcome(s) — CO:

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.

CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.

Bloom's Taxonomy Level

L3 – Apply, L4 – Analyze

SDG Mapping (SDG 1–17)

SDG 4 – Quality Education

SDG 8 – Decent Work and Economic Growth

SDG 9 – Industry, Innovation and Infrastructure

A. Problem Statement

A Smart National Exam Slot Booking and Result Management System is designed to manage large volumes of examination data including candidate details, exam schedules, centers, slot bookings, exam attempts, question-wise responses, and results. As the number of candidates and examination centers grows, inefficient storage and retrieval mechanisms can make candidate-history and center-wise audit queries slow.

At the same time, online slot booking and result processing can involve multiple users accessing the same records simultaneously. If booking and result updates are implemented as uncoordinated read-then-write operations, the system may experience double-booked slots, lost updates, inconsistent seat counts, or partially completed operations.

The proposed system addresses these problems through an integrated relational database solution using appropriate file organization, indexing, hashing, query optimization, ACID transactions, and concurrency-control techniques.

B. Objective

- Design a scalable relational database for national-level examination slot booking and result management.
- Provide efficient candidate-wise and center-wise retrieval of examination attempt records.
- Apply suitable file organization, indexing, and hashing techniques for efficient storage and lookup.
- Analyze and optimize important SQL queries using execution-plan analysis.
- Implement transaction-safe exam-slot booking using ACID properties, COMMIT, ROLLBACK, and SAVEPOINT.

- Prevent double booking when multiple candidates attempt to reserve the same final available slot.
- Ensure consistent result processing when multiple examiners update results concurrently.
- Evaluate performance, correctness, data integrity, security, and scalability using representative test data.

C. Requirements and Environment Used

- Frontend: HTML5, CSS3, JavaScript, Bootstrap 5 and Chart.js.
- Backend: Python 3 with Flask REST API.
- Database: MySQL 8.x.
- Database Tool: MySQL Workbench.
- Development Environment: Visual Studio Code.
- Diagram Tool: draw.io or equivalent diagramming software.
- Version Control: Git and GitHub.
- API Testing: Postman or browser-based API testing.
- The system must use a relational DBMS supporting indexes and transaction management.
- Database credentials must be protected using environment variables and must not be exposed in frontend code.
- Representative sample data should be used for meaningful performance and concurrency testing.

D. Design / Proposed Solution

System Architecture

Browser → Frontend → Flask REST API → MySQL Database

The system is divided into a presentation layer, application/backend layer, and relational database layer.

Major Modules

- User Authentication and Role Management
- Candidate Management
- Exam Management
- Examination Center Management
- Exam Slot Management
- Slot Booking
- Exam Attempt and Response Management
- Examiner and Result Management
- DBMS Analysis Dashboard

Database Tables

- users(user_id, username, password_hash, role)
- candidates(candidate_id, name, email, phone, date_of_birth)
- exam_centers(center_id, center_name, location, capacity)
- exams(exam_id, exam_name, description, duration)
- exam_slots(slot_id, exam_id, center_id, exam_date, start_time, end_time, capacity, available_seats)
- bookings(booking_id, candidate_id, slot_id, booking_time, status)
- exam_attempts(attempt_id, candidate_id, exam_id, center_id, attempt_date, start_time, end_time, score)

- attempt_responses(response_id, attempt_id, question_no, candidate_answer, correct_answer, time_taken)
- examiners(examiner_id, name, email)
- results(result_id, attempt_id, examiner_id, score, grade, status, published_at)

Storage and Retrieval Design

The proposed design uses indexed-sequential organization for exam-attempt retrieval, supported by primary and secondary indexes for common candidate-wise and center/date-wise queries. A separate educational hash-based lookup module demonstrates direct candidate/attempt lookup using a defined hash function and separate chaining for collision resolution.

Indexing Strategy

- B+-tree-style relational indexes are used for candidate identifiers and frequently filtered attributes.
- A composite index on center and attempt date supports center-wise audit queries.
- Candidate-based indexes support fast attempt-history and booking-history retrieval.
- The effect of indexing is evaluated using EXPLAIN/EXPLAIN ANALYZE and measured query performance.

Hashing Strategy

- Candidate IDs are mapped to buckets using a defined hash function.
- Collisions are handled using separate chaining.
- The demonstration reports the hash value, bucket number, collision status, and lookup result.
- The hashing demonstration is kept conceptually separate from MySQL B+-tree indexing.

Transaction and Concurrency Design

Slot booking is implemented as a bounded database transaction: BEGIN TRANSACTION → check and lock slot availability → create booking → update available seats → COMMIT. If any step fails, the transaction is rolled back. SAVEPOINT is used to demonstrate partial rollback. Appropriate row-level locking and transaction isolation are used to prevent double booking and inconsistent seat counts.

Result processing uses transaction and concurrency-control mechanisms to prevent lost updates when multiple examiners process the same result concurrently.

E. Algorithm / Pseudocode / Flowchart

1. Candidate Attempt Lookup

1. START
2. Receive Candidate ID
3. Use candidate index/hash lookup
4. Retrieve matching attempt records
5. Display candidate attempt history
6. END

2. Slot Booking

7. START
8. Receive candidate and slot information

9. BEGIN TRANSACTION
10. Lock/check the selected slot
11. Check whether available seats are greater than zero
12. If no seats are available, ROLLBACK and reject booking
13. Otherwise insert booking record
14. Decrease available seat count
15. COMMIT transaction
16. Return booking confirmation
17. END

3. Query Optimization

18. Select a representative unoptimized query
19. Execute EXPLAIN/EXPLAIN ANALYZE
20. Identify scans, joins, or filtering operations causing unnecessary work
21. Create appropriate indexes or rewrite the query
22. Execute the optimized query
23. Compare execution plans and actual measurements
24. Record the improvement

4. Concurrency Test

25. Create a slot with one remaining seat
26. Start two concurrent booking requests
27. Apply transaction and locking strategy
28. Allow only one transaction to reserve the final seat
29. Reject or safely fail the competing transaction
30. Verify that the final seat count is consistent
31. Record the result

F. Implementation / Source Code

The implementation consists of a responsive web frontend, Flask REST backend, and MySQL database. The backend uses parameterized SQL queries and explicit transaction handling. The frontend communicates with the backend through REST endpoints and does not directly access MySQL.

Important implementation components include:

- MySQL schema and relational constraints.
- Sample-data generation for performance testing.
- Primary and secondary indexes.
- Hashing demonstration with collision handling.
- SQL queries for candidate, center, booking, attempt, and result reporting.
- EXPLAIN/EXPLAIN ANALYZE based query analysis.
- Transaction-safe booking and result-processing logic.
- Concurrency-control demonstration.
- Flask REST API integration.
- Responsive frontend dashboard.

Complete source code should be maintained in the project's GitHub repository.

G. Test Cases and Expected / Actual Results

Test Case	Expected Result	Actual Result	Status
Lookup candidate attempt by Candidate ID	Matching attempt history is retrieved efficiently	To be recorded during execution	Pending
Center-wise audit for a selected date	All matching attempts are returned	To be recorded during execution	Pending
Hash lookup	Correct bucket and candidate/attempt record are returned	To be recorded during execution	Pending
Hash collision	Collision is detected and resolved using chaining	To be recorded during execution	Pending
Book available slot	Booking succeeds and available seats decrease	To be recorded during execution	Pending
Book full slot	Booking is rejected and data remains unchanged	To be recorded during execution	Pending
Two candidates book the last seat simultaneously	Only one booking succeeds; capacity is not exceeded	To be recorded during execution	Pending
Rollback booking transaction	No partial booking data remains	To be recorded during execution	Pending
SAVEPOINT demonstration	Changes after savepoint can be selectively rolled back	To be recorded during execution	Pending
Concurrent result update	Result remains consistent without an incorrect lost update	To be recorded during execution	Pending
Query before/after optimization	Optimized query shows improved execution characteristics	To be recorded during execution	Pending

H. Execution Screenshots / Output

The final report should include screenshots captured from the actual working implementation. Suggested evidence:

- Database and table creation.
- ER diagram and system architecture.
- Sample data loaded into MySQL.
- Index creation and index metadata.
- EXPLAIN/EXPLAIN ANALYZE before optimization.
- EXPLAIN/EXPLAIN ANALYZE after optimization.
- Hash lookup and collision demonstration.
- Transaction BEGIN/COMMIT.
- ROLLBACK and SAVEPOINT demonstration.
- Two-session concurrent booking test.
- Concurrent result-processing test.
- Frontend dashboard.
- Candidate, exam, center and slot pages.

- Successful and rejected booking outputs.
- Result management output.
- GitHub repository containing complete project files.

I. Analysis and Discussion

The proposed system is designed to improve retrieval efficiency while preserving consistency under simultaneous access. Indexed retrieval reduces unnecessary scanning for candidate-history and center-wise audit queries. Hashing provides an educational direct-lookup mechanism with near-constant average lookup behavior under suitable distribution and collision handling.

Query optimization is evaluated through execution plans and measured results rather than assumed performance. The transaction design ensures that booking and seat-count changes succeed or fail as one logical unit. Concurrency control prevents two users from consuming the same final seat.

The main trade-off is that additional indexes improve read performance but add storage and insert/update maintenance cost. Stronger transaction isolation or locking improves consistency but can increase waiting and reduce throughput under extreme contention. These trade-offs should be evaluated using the actual benchmark and concurrency results obtained during execution.

J. Conclusion

The Smart National Exam Slot Booking and Result Management System integrates database storage, efficient retrieval, query processing, optimization, and transaction management into one working application. The proposed indexed-sequential approach, indexing strategy, and hashing demonstration address the retrieval requirements, while optimized SQL and transaction-safe workflows address booking and result-processing consistency.

The final conclusion should be supported by actual execution-plan results, performance measurements, and concurrency-test evidence. The implementation should demonstrate that no slot is booked beyond capacity, no booking transaction is partially committed, and concurrent result processing maintains consistent data.

K. Individual Contribution of Group Member / Student

As this is a single-student implementation, the contribution write-up should describe the student's actual work across database design, SQL implementation, backend/frontend integration, testing, debugging, documentation, GitHub management, and final demonstration. The write-up should be approximately one page and should contain only truthful activities actually performed.

L. References

32. A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.
33. MySQL, MySQL 8.0 Reference Manual, Oracle Corporation.
34. Flask Documentation, Pallets Projects.
35. Bootstrap Documentation, Bootstrap Team.
36. draw.io Documentation, diagrams.net.
37. GitHub Documentation.

38. Any AI-assisted tools actually used during development, testing support, documentation, or refinement should be acknowledged according to the assessment instructions.

M. One-Page Write-up

The Smart National Exam Slot Booking and Result Management System was developed as a practical DBMS application to combine efficient data storage and retrieval with reliable transaction processing. The project models candidates, examinations, centers, slots, bookings, exam attempts, responses, examiners, and results in a relational MySQL database.

The implementation applies indexed-sequential storage principles and secondary indexes to support candidate-history and center-wise retrieval. A separate hashing demonstration explains direct lookup and collision resolution through chaining. SQL execution plans are examined before and after optimization to identify unnecessary scans and improve query performance.

The booking module is implemented using database transactions so that checking availability, creating a booking, and updating the available-seat count are treated as one atomic operation. COMMIT, ROLLBACK, and SAVEPOINT are demonstrated. Concurrency testing is used to verify that simultaneous candidates cannot book the same final seat and that result processing does not suffer from incorrect lost updates.

The project also provides a professional web interface connected to a Flask backend and MySQL database. Testing, screenshots, execution plans, transaction evidence, and measured performance results are included to validate the implementation. The overall solution demonstrates how DBMS concepts can be applied to build an efficient, consistent, secure, and scalable examination management system.

N. Submission Details

- Use the prescribed assignment document format.
- Do not place register number or name inside the main body if the faculty's general instruction prohibits it; follow the latest faculty instruction for the final submission.
- Include the required title and team/member information at the beginning if specifically required by the faculty.
- Include the student's register number and name at the end under 'Submitted By' if required by the faculty.
- Submit one PDF/soft copy as instructed.
- Upload the complete working project/code to GitHub.
- Ensure plagiarism percentage is below 20% as specified by the faculty.
- Use actual execution screenshots and actual measured results; do not fabricate evidence.

Submitted By:

Name: SANTHOSH.E (192511470) / HARSHITH KARNAM (192511384)